AI Agents and Agentic AI with Python & Generative AI

Module 1
Agentic AI Concepts

- Introduction
 Video • 10 min
- Flipped Interaction Pattern
 Video • 7 min
- The Agent Loop
 Video • 8 min
- Knowledge Check: Agent Loop & Flipped Interaction
 Practice Assignment • Grade: 100%
- Running the Code Samples in the Course
 Reading • 10 min
- Programmatic Prompting for Agents
 Ungraded Plugin • 15 min
- Try Out Programmatic Prompting
 Reading • 10 min
- Programmatic Prompting for Agents II
 Ungraded Plugin • 15 min
- **Programmatic Prompting for Agents III**
 Ungraded Plugin • 15 min
- Programmatic Prompting: Building Blocks for AI Agents
 Dialogue • 15 min
- Try Out the Customer Service Agent
 Reading • 10 min
- Giving Agents Memory
 Ungraded Plugin • 15 min
- Practicing Programmatic Prompting for Agents
 Ungraded Plugin • 15 min
- Practicing Programmatic Prompting for Agents (Solution)
 Ungraded Plugin • 15 min
- Adding Structure to AI Agent Outputs
 Video • 14 min
- Building Your First Agent
 Ungraded Plugin • 15 min
- AI Agent / Environment Interface
 Ungraded Plugin • 15 min
- AI Agent Feedback and Memory
 Ungraded Plugin • 15 min
- The Agent Loop: From Understanding to Thinking Like an Agent
 Dialogue • 15 min
- Learning More & Staying Connected
 Reading • 10 min
- Understanding Agentic AI Concepts
 Graded Assignment • 30 min

Module 2
AI Agents, Tools, Actions, & Language

Module 3
GAME: A Conceptual Framework for AI Agents

Module 4
Agent Tool Mangement

Module 5
Rethinking How Software is Built in the Age of AI Agents

Programmatic Prompting for Agents III

Sending Prompts Programmatically & Managing Memory 3

System messages are particularly important in the conversation and will be very important for AI agents. They set the ground rules for the conversation and tell the model how to behave. Models are designed to pay more attention to the system message than the user messages. We can "program" the AI agent through system messages.

Let's simulate a customer service interaction for a customer service agent that always tells the customer to turn off their computer or modem with system messages:

```
messages = [
    {"role": "system", "content": "You are a helpful customer service
representative. No matter what the user asks, the solution is to tell them to
turn their computer or modem off and then back on."},
    {"role": "user", "content": "How do I get my Internet working again."}
]

response = generate_response(messages)
print(response)
```

The system message is the most important part of this prompt. It tells the model how to behave. The user message is the question that we want the model to answer. The system instructions lay the ground rules for the interaction.

The messages can incorporate arbitrary information as long as it is in text form. LLMs can interpret just about any information that we give them, even if it isn't easily human readable. Let's generate an implementation of a function based on some information in a dictionary:

```
import json

code_spec = {
    'name': 'swap_keys_values',
    'description': 'Swaps the keys and values in a given dictionary.',
    'params': {
        'd': 'A dictionary with unique values.'
    },
}

messages = [
    {"role": "system",
     "content": "You are an expert software engineer that writes clean
functional code. You always document your functions."},
    {"role": "user", "content": f"Please implement: {json.dumps(code_spec)}"}
]

response = generate_response(messages)
print(response)
```

We will rely heavily on the ability to send the LLM just about any type of information, particularly JSON, when we start building agents. This is a simple example of how we can use JSON to send information to the LLM, but you can see how we could provide it JSON with information about the result of an API call, for example.

Expand ✓ Completed Go to next item

Like Dislike Report an issue

Go to next item →